

Chidera's Deliverables

What Chidera owns, what is finished, what is still to do, and the number that says each item is done. The endpoints Paul's dashboard depends on are in the companion document, *Dashboard API Contract*.

Chidera owns everything between a payment arriving and a person seeing it: the intake service, the scoring queue and workers, the live measurements, the rules and policy code, the case system and response clocks, the recommendation engine, sign-in and roles, the permanent record, the database, and the API the dashboard reads. Ire supplies the data and the trained model; Paul builds the screens on top of this API. Chidera's job is that every payment is scored once, fast, reproducibly, and that the screens can only do what each role is allowed to do.

Where things stand

AREA	MEASURE	TARGET	NOW	STATUS
Speed	Payment to decision, 95% of the time (NFR-001)	under 2 s	0.88 s	PASS
Load	Payments lost in a five-times burst (NFR-002)	0	0	PASS
Workload	Alerts per day, whole system	≤ 120	82 (latest demo day)	MET
Record	Entries that can be edited or deleted	none	none · hash-chained, checkable	MET
Measurements	Same twelve numbers live as in training	identical	identical, tested	MET
Roles	Administrator can open a case	never	never (refused by the server)	MET
Sign-in	Two-step sign-in for lead and administrator	mandatory	mandatory	MET
Advice	Every case has an action, steps and a reason	100%	100%	MET
Advice	How often analysts follow the advice	reported weekly	recorded, not reported	NOT YET
Advice	Bank's own procedure loaded	yes	our playbook, replaceable	PARTLY
Dashboard	Endpoints the full dashboard needs	all	five gaps listed	PARTLY
Quality	Automated tests, against a real database	all pass	91 pass	MET

Finished — Chidera presents these at the defence

- **Intake.** Single and batch payment endpoints; a repeated reference is recognised as a duplicate, not a second payment. Identities are one-way hashed on arrival and never stored readable (D9c, D19a).
- **Queue and workers.** A database-backed queue with leases, so no payment is handed to two workers; a test proves it. Three workers keep up with a five-times burst without losing a payment.
- **Scoring.** Measurements are computed once, then go to the rules and the model at the same time; the policy step alone combines their answers (see the Blueprint diagram). Every decision stores the versions that produced it, so it can be reproduced months later.
- **Rules and policy.** Six rules of three powers (raise, override, lower). The payment-burst rule now looks at where money is going (D67). The whole-system alert budget spends rule alerts first and gives the model what is left.
- **Case system.** Alerts about one customer group into one case within 24 hours; response clocks per severity; a worklist ordered by severity, money and lateness; "take the next case" assigns it so two analysts never work the same one.
- **Recommendation engine.** One action, numbered steps and a reason for every case, from a written playbook; the analyst's choice of whether they followed it is saved with every outcome.
- **Sign-in, roles and the record.** Four roles, server-side sessions in a cookie scripts cannot read, two-step sign-in, and a permanent record where every entry is chained to the last; a check endpoint proves nothing was altered.
- **Live alerts.** A stream that replays anything missed after a dropped connection, so the dashboard never loses an alert.
- **Demo builder.** Rebuilds a believable desk in about ten minutes; outcomes follow the simulator's truth (D66a) and the live day continues the same bank (D68).

Still to do

1 Close the five API gaps the full dashboard needs AGREED WITH PAUL	
Why	The shareable Risk Radar Desk design has views the current API cannot fully feed. Each gap, with its proposed endpoint, is in the <i>Dashboard API Contract</i> .
What	Advice-followed rate on the operations metrics; the model's evaluation figures for the administrator; an "escalated to" filter for information security; dormancy on worklist items; "turned into alerts" counts per rule.
Done when	Each field exists, has a test, and Paul confirms the screen reads it with no workaround.

2 Report how often analysts follow the advice NOT YET

Why	It is the only honest check on whether the recommendation engine is any good. The answer is already saved with every outcome; nothing reads it back.
What	Add to GET /v1/metrics/operations: share of decided cases where the advice was followed, overall and per recommended action.
Done when	The ops-lead view shows the rate for the last seven days, and the Blueprint's "not yet" becomes a number.

3 Put the team's alert-budget choice live WAITING ON THE TEAM

Why	A smaller budget cuts false alarms sharply now that the burst rule is precise: about 75 a day at 120 alerts, 33 at 75, 20 at 60, at some cost to never-seen account takeover (D67c).
What	Once chosen: set the budget, run scripts/derive_thresholds.py --budget N --publish for a new audited threshold set, rebuild the demo, and update the figures in every document.
Done when	The live threshold set records the new budget, the demo day stays within it, and the decision is logged.

4 Make the bank's procedure loadable SLOT BUILT

Why	No bank has shared its fraud procedures, so the playbook is our own. It must be replaceable without code, and never described as a bank's policy until one is loaded.
What	Document the playbook's format (situation in, action, steps and urgency out) and load it from a file the administrator can replace, with every change recorded.
Done when	Swapping the file changes the recommendations on the next case, with no deployment, and the change appears in the permanent record.

5 Keep the contract and the code in step ONGOING

Why	A field renamed on the server breaks a screen silently.
What	A contract test that calls each endpoint in the API contract and checks the fields listed there; any change to a field updates the contract document in the same commit.
Done when	The test runs with the others, and the contract is frozen on the Monday of week five (feature freeze).

Rules Chidera works under

- **Advisory only.** Nothing Risk Radar does blocks, holds or reverses a payment.
- **The server enforces roles.** The dashboard hides what a role cannot do, but the API refuses it regardless.
- **Money is integer kobo everywhere,** in the database, the API and every total.
- **Decisions and alerts are never edited;** only cases change, and every change is recorded.
- **Detection never reads the simulator,** and a test enforces it.

Sources: docs/qa-report.md (NFR-001, NFR-002), backend/riskradar/api/, backend/riskradar/security/rbac.py, decisions D9-D68 in docs/decisions.md. Companions: Dashboard API Contract, Ire's Deliverables, Risk Radar Blueprint.